Knowledge-Based Systems 339 (2026) 115554

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys

MRDNet: Multivariable Relational Decomposition Network for Multivariate Time Series Forecasting

Ao Hu [a], Liangjian Wen [a,1,*], Mingyi Zhang [b], Yong Dai [c,d], Dongkai Wang [a], Jun Wang [e], Jiang Duan [a]

[a] School of Computing and Artificial Intelligence, Southwestern University of Finance and Economics, Chengdu, China
[b] The College of Computer Science, Sichuan University, Chengdu, China
[c] The X-Humanoid Research Institute, Beijing, China
[d] Artificial Intelligence and Digital Finance Key Laboratory of Sichuan Province, Chengdu, China
[e] School of Management Science and Engineering, Southwestern University of Finance and Economics, Chengdu, China

ARTICLE INFO

Keywords:
Multi-variable time series forecasting (MTSF)
Time series decomposition

ABSTRACT

Accurate modeling of correlations among variables (also referred to as channels) is essential for achieving precise and reliable multivariate time series forecasting. However, challenges persist in multivariate time series analysis due to anomalies such as outliers, missing values, and noise, which obscure true relationships between variables and reduce forecasting accuracy. To address this, we propose a Multivariate Relational Decomposition Network (MRDNet) that separates variable relationships into global and local components. The global component captures shared seasonal patterns among variables using learnable latent vectors, establishing stable long-term correlations. The local component focuses on window-specific relationships by analyzing trends in smoothed time series data. Additionally, we design an adaptive neural network to refine these relationships and mitigate the effects of anomalies. By integrating these components, MRDNet effectively handles anomalies while capturing both stable global dependencies and dynamic local patterns. Extensive experiments on benchmark datasets demonstrate that MRDNet achieves superior forecasting performance, addressing challenges posed by multivariate time series anomalies.

1. Introduction

Time series forecasting is extensively applied across multiple fields, including transportation [1,2], financial markets [3,4], and meteorological predictions [5,6]. Recent progress in deep learning techniques [7–9] has significantly enhanced prediction accuracy. Notably, Transformer-based architectures [10–13] have demonstrated exceptional proficiency in capturing extended temporal relationships and managing complex cross-temporal interactions.

Recent research has increasingly focused on modeling cross-variable dependencies to enhance the precision of time series forecasting. Previous studies, such as those introducing STGCN [14], MTGNN [15], and CrossGNN [16], utilized Graph Neural Networks (GNNs) to explicitly represent inter-variable relationships. Additionally, models like Informer [10], Autoformer [11], and FEDformer [12] adopt an implicit approach by encoding multivariate time series data into temporal tokens. This strategy, however, may introduce noise from misaligned temporal tokens, which can compromise the accurate capture of cross-variable interactions. To address this limitation, Crossformer [17] employs a dual-stage attention mechanism that simultaneously considers variables and temporal segments, thereby improving the modeling of multivariate correlations. More recently, iTransformer [18] has advanced this methodology by treating each time series as a distinct token and applying self-attention to directly model inter-variable dependencies, demonstrating enhanced performance on high-dimensional datasets.

Despite recent advances, identifying meaningful relationships within multivariate time series poses significant difficulties. Multivariate time series data often contain outliers, missing values, and noise, as shown in Fig. 1. These anomalies obscure variable relationships. For example, an extreme outlier might inflate a variable's attention score, causing the model to overemphasize spurious connections rather than genuine dependencies. This interference can ultimately reduce prediction accuracy, posing a significant challenge for effective time series modeling.

To tackle this issue, we propose a decomposition-based approach that separates multivariate relationships into global and local components, directly addressing the adverse effects of anomalies while cap-

* Corresponding author.
E-mail address: wlj6816@gmail.com (L. Wen).
[1] Liangjian Wen On behalf of all co-authors.

https://doi.org/10.1016/j.knosys.2026.115554
Received 7 July 2025; Received in revised form 18 January 2026; Accepted 12 February 2026
Available online 18 February 2026

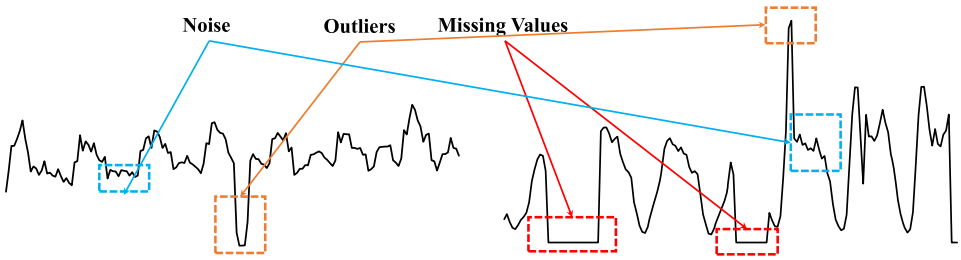

Fig. 1. Instance visualization of training sets for ETTh1 (left) and Traffic (right). Time series data often exhibit anomalies such as outliers, missing values, and noise, which complicate variable relationships.

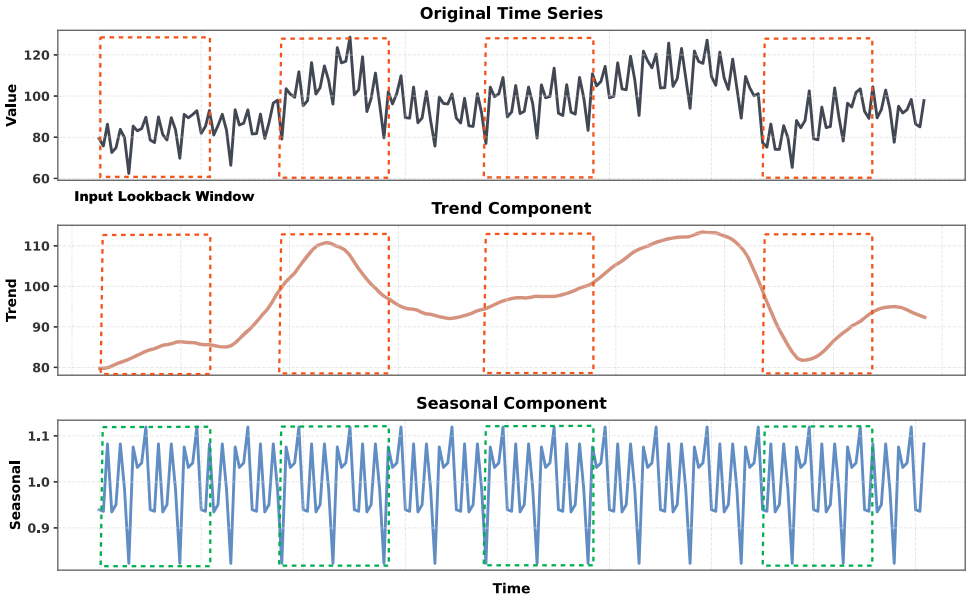

Fig. 2. Visualization of an original multivariate time series and its decomposed seasonal and trend components within input windows. In the Seasonal component, variables exhibit periodic changes, displaying a stable global relationship. And in the Trend component, due to distribution drift, dynamic local relationships are observed within the input window.

turing both stable and dynamic patterns among variables. The global component captures the shared seasonal patterns across all variables. As shown in Fig. 2 (top: original series; middle: smoothed trend; bottom: seasonal), these seasonal patterns exhibit periodic fluctuations that remain consistent over the entire time series, thereby emphasizing enduring, stable relationships among the variables. By representing common seasonal effects that influence all variables similarly, this global component engenders long-term, stable correlations. To account for variable-specific deviations and mitigate the impact of anomalies, we allocate a learnable latent vector to each variable. The learnable nature of these vectors facilitates adaptive modeling, enabling the capture of optimal representations of both the shared seasonal relationships and the individual inter-variable relationships.

In contrast, the local component captures the trend relationships among all variables specific to particular input windows. We compute these window-specific variable relationships by analyzing the trend of time series data, which is computed through smoothing the sequences within each respective input window (as visualized in the middle panel of Fig. 2). To further mitigate the influence of anomalies, such as abrupt spikes or missing data points, we implement a neural network architecture that adaptively refines the window-specific variable relationships. This network acts as a filter on the initially computed variable relationships, which may contain noise, thereby producing more robust representations of the underlying temporal dynamics.

By integrating these strategies, we propose a multivariate relational decomposition network (MRDNet) approach, which decomposes relationships into global and local components, further refining them through seasonal and trend partitioning. This methodology effectively addresses the challenges arising from outliers, missing values, and noise, while simultaneously capturing both stable, long-term dependencies and dynamic, short-term fluctuations inherent in multivariate time series data, thereby enhancing the accuracy and robustness of predictions. Our contributions are as follows:

- We identify and address the challenges caused by anomalies, including outliers, missing values, and noise, in multivariate time series data that obscure true variable dependencies and undermine forecasting accuracy.
- We create a dual-path learning system combining global and local elements. The global path uses learnable vectors to model seasonal patterns and variable correlations, while the local path employs adaptive networks for refined trend analysis. This can capture both stable and dynamic patterns while managing data issues.
- We integrate these strategies into a unified MRDNet architecture that models global and local relational structures. Extensive experiments on benchmark multivariate time series datasets demonstrate that MRDNet achieves superior forecasting performance.

2. Related Wwork

Deep learning models, including Transformer-based [10–13,17,18], MLP-based [8,19–21], graph neural network (GNN)-based, and convolu-

tional neural network (CNN)-based models [9,22–24], demonstrate significant efficacy in time series forecasting. These models enhance accuracy by capturing temporal and multivariate dependencies. In this section, we categorize them into two main groups: cross-temporal interaction models (further divided into decomposition-based and non-decomposition-based models) and cross-variable interaction models.

2.1. Cross-temporal interaction models

2.1.1. Decomposition-based models

Decomposing time series into components facilitates modeling long-term dependencies. For example, Prophet [25] uses trend-seasonality decomposition, N-BEATS [26] employs basis decomposition, and DeepGLO [27] applies matrix factorization. Autoformer [11] integrates decomposition into deep learning frameworks for hierarchical effects, while FEDformer [12] enhances Transformers with frequency-based trend-seasonality decomposition. DLinear [8] uses a dual-branch linear layer for similar decomposition, questioning Transformer efficacy in multivariate forecasting. S^2IP-LLM [28] leverages large language models to capture seasonal, trend, and residual components. However, these models often neglect complex multivariate relationships. Our proposed MRDNet addresses this by capturing both global and local variable interactions through decomposition, significantly improving forecasting performance.

2.1.2. Non-decomposition-based models

Transformer-based models, such as Informer [10] and Pyraformer [29], optimize long-range temporal dependency modeling while reducing attention mechanism costs. However, simpler linear models like NLinear [8], RLinear [20] and TimeMixer [30], Which use MLPs to capture periodic patterns, often outperform complex Transformer-based models. PatchTST [13] employs variable-independent modeling across time patches. CNN-based models, including MICN [22] and TimesNet [23], show competitive performance. Recent research has also increasingly explored frequency-domain approaches, such as FreTS [31], FITS [32], and FilterNet [21].

2.2. Cross-variable interaction models

Modeling cross-variable interactions is critical for accurate time series forecasting. GNN-based models, such as STGCN [14] and MTGNN [15], effectively capture spatial and temporal variable dependencies. CNN-based ModernTCN [24] uses ConvFFN1 for independent feature learning and ConvFFN2 for cross-variable dependencies. Transformer-based Crossformer [16] employs a two-stage attention mechanism to model both temporal and variable dependencies, enhancing accuracy. Similarly, iTransformer treats time series as variable tokens, using explicit attention for state-of-the-art performance on high-dimensional datasets. TimeXer [33] introduces external variables to enhance multivariate modeling relationships. However, variable relationships are easily affected by outliers, missing values, or noise, so capturing more robust cross-variable relationships is still a challenge.

3. Proposed method

3.1. The general structure

This work addresses the multivariate time series forecasting problem. Let $\mathbf{X} \in \mathbb{R}^{N\times L}$ represent the historical observations, where N denotes the number of variables and L represents the length of the historical sequence. Our objective is to predict the future T time steps, represented as $\mathbf{Y} \in \mathbb{R}^{N\times T}$. The architecture of the proposed method is depicted in Fig. 3. This approach employs an encoder-only architecture, integrating essential components, including the Decomposition Block, Seasonal Modeling Block, Trend Modeling Block, and a summation module for prediction.

3.2. Decomposition block

Long-term time series data exhibit distinct seasonal and trend components. Prior studies [8,11,12] have shown that decomposing time series into trend and seasonal subsequences enhances the understanding of complex temporal patterns. Furthermore, we observed that the seasonal component exhibits stable variable relationships, whereas the trend component displays variability, necessitating distinct processing approaches. There are two primary methods for time series decomposition. The first employs a moving average to smooth the time series, emphasizing long-term trends, with the seasonal component derived by subtracting the estimated trend from the normalized time series. The second method, Seasonal-Trend Decomposition using LOESS (STL) [34], is computationally intensive for multivariate time series, leading us to favor the first approach. For an input time series $\mathbf{X} \in \mathbb{R}^{N\times L}$ of length L, the decomposition is formulated as:

$$\mathbf{X}_t = \text{AvgPool}(\text{Padding}(\mathbf{X})), \tag{1}$$

$$\mathbf{X}_s = \mathbf{X} - \mathbf{X}_t, \tag{2}$$

where $\mathbf{X}_s, \mathbf{X}_t \in \mathbb{R}^{N\times L}$ represent the seasonal and trend components of input time series, respectively.

3.3. Seasonal modeling block

The relationship among variables in the seasonal component stabilizes due to its periodicity, which allows us to model inter-variable dependencies robustly without interference from window-specific anomalies. To capture the intrinsic nature of each variable, we introduce a group of globally learnable latent vectors $\mathbf{V} \in \mathbb{R}^{N\times d}$. Instead of relying on the variable relationships within the input window of the seasonal component, which may be affected by noise, outliers, or missing values, we utilize the relationships among the latent vectors. Specifically, we compute the Gramian matrix $\mathbf{R}_V$ of the latent vectors to represent the variable relationships of the input $\mathbf{X}_s$, thereby capturing more robust dependencies. The formulation is as follows:

$$\mathbf{R}_V = \mathbf{V} \cdot \mathbf{V}^T, \tag{3}$$

$$\hat{\mathbf{X}}_s = \mathbf{R}_V \cdot \mathbf{X}_s, \tag{4}$$

where $\mathbf{R}_V \in \mathbb{R}^{N\times N}$, $\hat{\mathbf{X}}_s \in \mathbb{R}^{N\times L}$, and $\cdot$ denotes matrix multiplication. After capturing more stable variable relationships, we fuse the information from $\mathbf{V}$, $\hat{\mathbf{X}}_s$, and $\mathbf{X}_s$, embedding them into a shared representation space to learn robust variable representations. These representations are further refined using feedforward neural networks (FFN) to capture future seasonal patterns. The process is formalized as follows:

$$\mathbf{X}_{s,\text{emb}} = \text{Linear}(\text{Concatenate}(\mathbf{V}, \hat{\mathbf{X}}_s, \mathbf{X}_s)), \tag{5}$$

$$\mathbf{X}_{s,\text{ffn}} = \text{FFN}(\mathbf{X}_{s,\text{emb}}), \tag{6}$$

where $\mathbf{X}_{s,\text{emb}}, \mathbf{X}_{s,\text{ffn}} \in \mathbb{R}^{N\times D}$, and D represents the feature dimension. Finally, a projection layer maps the learned seasonal representations to the seasonal components:

$$\hat{\mathbf{Y}}_s = \text{Projection}(\mathbf{X}_{s,\text{ffn}}), \tag{7}$$

where $\hat{\mathbf{Y}}_s \in \mathbb{R}^{N\times T}$ represents the future seasonal patterns.

3.4. Trend modeling block

In multivariate time series forecasting, the seasonal component exhibits stable dependencies, effectively captured by the Gramian matrix of global latent vectors. In contrast, the trend component displays stronger local dependencies due to its non-stationary nature, necessitating optimization of variable relationships within the input window. This variability requires window-specific optimization to filter anomaly-induced noise while adapting to dynamic patterns. To address this, we propose a method that computes Gramian matrix $\mathbf{R}_{ac} \in \mathbb{R}^{N\times N}$ for each pair of

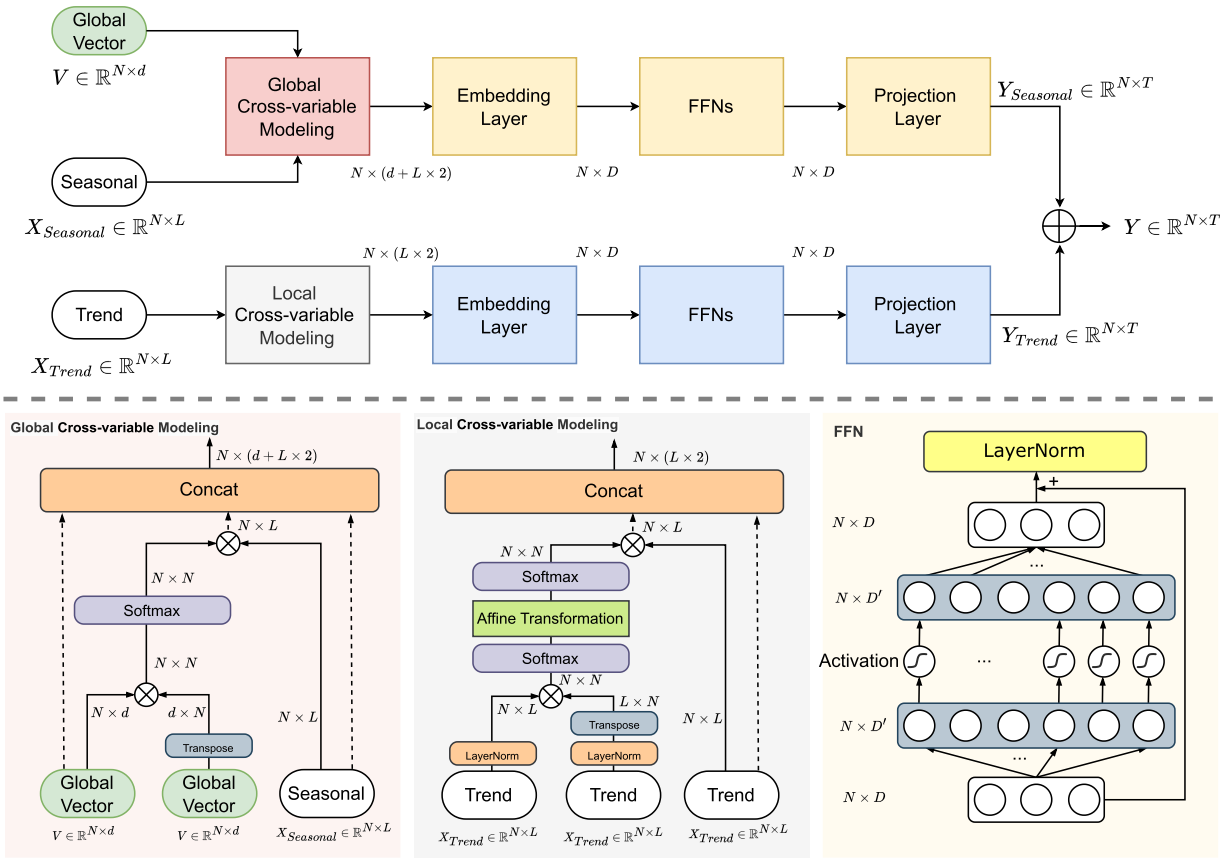

Fig. 3. Overview of the MRDNet architecture, which processes decomposed seasonal and trend inputs through parallel branches. It includes Global Cross-variable Modeling (GCM) block, Local Cross-variable Modeling (LCM) block, Embedding Layers, Feed-Forward Networks (FFNs), and Projection Layers for final prediction. Global variable relationships are modeled in the Seasonal branch, while local variable relationships are captured in the Trend branch.

variables in $\mathbf{X}_t \in \mathbb{R}^{N\times L}$, representing their pairwise relationships. We further introduce learnable weight parameters $W \in \mathbb{R}^{N\times N}$ and bias terms $b \in \mathbb{R}^N$ to adaptively refine variables' relationship, producing a tailored matrix that better adjusts the similarity among variables. The detailed procedure is outlined in the following steps:

$$\mathbf{X}_{norm} = \text{LayerNorm}(\mathbf{X}_t), \tag{8}$$

$$\mathbf{R}_{ac} = \mathbf{X}_{norm} \cdot \mathbf{X}_{norm}^T, \tag{9}$$

$$\mathbf{R}_{sm} = \text{Softmax}(\text{Softmax}(\mathbf{R}_{ac}) \cdot W^T + b), \tag{10}$$

$$\hat{\mathbf{X}}_t = \mathbf{R}_{sm} \cdot \mathbf{X}_t, \tag{11}$$

where $\mathbf{X}_t, \mathbf{X}_{norm}, \hat{\mathbf{X}}_t \in \mathbb{R}^{N\times L}$, $\mathbf{R}_{ac}, \mathbf{R}_{sm} \in \mathbb{R}^{N\times N}$, and $\cdot$ denotes matrix multiplication. Given the input trend component $\mathbf{X}_t$, we first apply layer normalization to each variable. Subsequently, we compute the Gramian matrix among variables and transform it into a similarity matrix using the softmax function. This matrix is then modulated by learnable weights to adaptively adjust inter-variable similarities. Finally, the softmax operation is reapplied, and the output is multiplied by $\mathbf{X}_t$ to incorporate variable dependencies into the time series.

After refining the variable relationships within the input window, we integrate the refined relationships into $\hat{\mathbf{X}}_t$ and embed it with the original input $\mathbf{X}_t$. A linear layer with shared weights projects both into a common vector space to further refine the variable representations. Subsequently, a stacked feedforward neural network (FFN) processes the embedded representations to learn future variable representations, as follows:

$$\mathbf{X}_{t,emb} = \text{Linear}(\text{Concatenate}(\hat{\mathbf{X}}_t, \mathbf{X}_t)), \tag{12}$$

$$\mathbf{X}_{t,ffn} = \text{FFNs}(\mathbf{X}_{t,emb}), \tag{13}$$

where $\mathbf{X}_{t,emb}, \mathbf{X}_{t,ffn} \in \mathbb{R}^{N\times D}$, and D denotes the feature dimension. Finally, a projection layer maps the learned trend representations to the trend within the future window:

$$\hat{\mathbf{Y}}_t = \text{Projection}(\mathbf{X}_{t,ffn}), \tag{14}$$

where $\hat{\mathbf{Y}}_t \in \mathbb{R}^{N\times T}$. This projection enables the model to predict future trends with the refined representations.

Feed-forward network. Leveraging the strengths of prior MLP-based models [8,20], we adopt the Feed-Forward Network (FFN) from the Transformer architecture to enhance variable representation learning. The FFN comprises two fully connected layers, an activation function, LayerNorm, and a residual connection. We employ the GELU activation function, with the structure defined as follows:

$$X = \text{LayerNorm}(X + \text{FC}(\text{GELU}(\text{FC}(X)))).$$

Prediction. After processing through the Seasonal Modeling Block and the Trend Modeling Block, the learned seasonal and trend components are combined to form the final predicted time series, $\hat{\mathbf{Y}}$, as follows:

$$\hat{\mathbf{Y}} = \hat{\mathbf{Y}}_s + \hat{\mathbf{Y}}_t, \tag{15}$$

where $\hat{\mathbf{Y}} \in \mathbb{R}^{N\times T}$ denotes the predicted future values, and the Mean Squared Error (MSE) serves as the loss function for training.

4. Experiments

4.1. Experimental settings

Datasets. To rigorously evaluate the efficacy of the proposed MRDNet, we conducted extensive experiments across 12 diverse real-world multivariate time series datasets. These include the following: ECL [11], Traffic [11], Weather [11], Solar-Energy [35], four PEMS datasets (PEMS03, PEMS04, PEMS07, PEMS08) [9] and four ETT datasets (ETTh1, ETTh2,

Table 1
Statistics of multivariate datasets. The description of each dataset includes the number of variables, the number of timesteps, the proportions of data allocated to training, validation, and test sets, and the sampling frequency of the data.

	ETTh1	ETTh2	ETTm1	ETTm2	Weather	ECL	Traffic	Solar-Energy	PEMS03	PEMS04	PEMS07	PEMS08
Variables	7	7	7	7	21	321	862	137	358	307	883	170
Timesteps	14,400	14,400	57,600	57,600	52,696	26,304	17,544	52,560	25,887	16,992	28,155	17,856
Proportion	6:2:2	6:2:2	6:2:2	6:2:2	7:1:2	7:1:2	7:1:2	7:1:2	6:2:2	6:2:2	6:2:2	6:2:2
Frequency	1 h	1 h	15 min	15 min	10 min	1 h	1 h	10 min	5 min	5 min	5 min	5 min
Domain	Electricity	Electricity	Electricity	Electricity	Weather	Electricity	Transportation	Energy	Transportation	Transportation	Transportation	Transportation

ETTm1, ETTm2) [10]. These datasets are commonly used in time series forecasting tasks, and we processed them according to the methodology outlined in iTransformer [18]. The details of the datasets are presented in Table 1.

Baselines and experimental settings. We evaluate our method against several state-of-the-art baseline models, including Transformer-based models: TimeXer [33], iTransformer [18], PatchTST [13], Crossformer [17], and FEDformer [12]. MLP-based models: FilterNer [21], TimeMixer [30], TSMixer [36], TiDE [19] and DLinear [8]. And CNN-based models: TimesNet [23]. For a fair comparison, we set the lookback window to 96 for all baseline models. The prediction windows are configured as follows: $\{12, 24, 48, 96\}$ for the PEMS datasets and $\{96, 192, 336, 720\}$ for other benchmarks. The results for MRDNet, FilterNet, iTransformer, TimeMixer, PatchTST and TSMixer are obtained from our experiments, while the results for the other baseline models are taken from the original iTransformer [18].

Metrics. Consistent with prior studies, we employ Mean Squared Error (MSE) and Mean Absolute Error (MAE) as metrics to evaluate the predictive performance of our model. Specifically, we denote the predicted value at time step i as $\hat{\mathbf{Y}}_\mathbf{i}$ and the corresponding true value as $\mathbf{Y}_\mathbf{i}$, with T representing the total number of prediction steps. The calculation formulas are as follows:

$$MSE = \frac{1}{T}\sum_{i=1}^{T}(\hat{\mathbf{Y}}_\mathbf{i} - \mathbf{Y}_i)^2, \quad MAE = \frac{1}{T}\sum_{i=1}^{T}\left|\hat{\mathbf{Y}}_\mathbf{i} - \mathbf{Y}_\mathbf{i}\right| \tag{16}$$

4.2. Main result

The quantitative results of Multivariate Time Series (MTS) forecasting are presented in Table 2. The proposed MRDNet consistently outperforms competing methods across diverse prediction horizons and datasets, achieving substantial reductions in Mean Squared Error (MSE) and Mean Absolute Error (MAE). Specifically, MRDNet achieves the lowest MSE in 43 cases and the lowest MAE in 44 cases across 12 real-world datasets, surpassing state-of-the-art (SOTA) models. In contrast, variable-independent models, which perform adequately on datasets such as ETT and Weather, falter due to their limited capacity to capture inter-variable dependencies. Notably, on datasets with numerical variables (e.g., ECL, Traffic, Solar-Energy, and PEMS), MRDNet outperforms iTransformer, which struggles to handle outliers and noise when modeling local variable relationships. This performance degradation is particularly evident in the ETT and Weather datasets, where fewer variables exacerbate the impact of anomalies, rendering iTransformer less effective than PatchTST. Conversely, MRDNet mitigates these challenges by effectively decomposing global and local variable relationships, thereby enhancing prediction accuracy through robust identification of true variable dependencies.

To further validate the temporal alignment quality of our predictions, we conduct additional comparisons using Dynamic Time Warping (DTW) distance, as shown in Table 3. MRDNet consistently achieves the lowest DTW distance across all datasets and prediction horizons, demonstrating superior capability in capturing temporal dynamics and maintaining sequence alignment with ground truth.

4.3. Visualization analysis of MRDNet

4.3.1. Visualization of GCM

To evaluate the effectiveness of the Global Cross-variable Modeling (GCM) in capturing relationships among global variables, we visualized the learned global latent vector V during the testing phase. Specifically, we applied t-SNE to project V onto a two-dimensional space which could preserve the local manifold structures of the high-dimensional latent vectors for the Weather (21 variables) and ECL (321 variables) datasets respectively, analyzing input windows comprising three variables. The results are shown in Fig. 4. On the Weather dataset, variables 4 and 10 appear closer in the t-SNE projection, with their time-series plots exhibiting greater similarity, whereas variable 17 is more distant due to inherent differences and missing values. Similarly, in the ECL dataset, variables 164 and 179 display high similarity in periodicity and trends, while variable 28 differs significantly due to intrinsic variations and noise perturbations. These visualizations confirm that the global latent vector effectively captures variable similarity relationships. The global vector adaptively learns to position variables with clear periodic patterns near the center of the distribution, while placing those severely affected by noise toward the periphery. This validates the GCM's efficacy in modeling global variable interactions.

4.3.2. Visualization of LCM

To evaluate the effectiveness of the Local Cross-variable Modeling (LCM) block, we visualized the attention map on the ECL dataset after applying affine transformation and softmax, alongside the distribution of variables in the predicted future sequence using principal component analysis (PCA), which was selected to reveal the global variance and dominant directions of the actual values. The results, shown in Fig. 5, reveal that the attention map processed by the LCM module exhibits a banded structure, where each column approximates the similarity between a given variable and others. Consequently, we identified the two variables with the highest column sums as 'center variables.' In the actual future sequence distribution, these variables were also centrally located, confirming that the LCM module effectively captures local variable relationships. This adaptive approach emphasizes key variable interactions, mitigates the impact of anomalous values and noise, and enhances the model's predictive performance.

4.4. Ablation study

To evaluate the efficacy of our proposed multivariate relationship decomposition method, we performed ablation studies by systematically removing or replacing key components: the Global Cross-variable Modeling (GCM) module and the Local Cross-variable Modeling (LCM) module. The 'w/o GCM' configuration employs only the LCM module to capture local variable relationships in the Seasonal and Trend branches, excluding global relationships. Similarly, the 'w/o LCM' configuration uses only the GCM module to capture global relationships in both branches, omitting local relationships. Additionally, we included the iTransformer [18] model as a baseline for comparison.

We selected ECL, Traffic, PEMS08, and ETTm1 as our benchmarks, with MSE results for four prediction lengths presented in Table 4. Experimental results demonstrate that MRDNet exhibits superior predictive

Table 2
Full results for the multivariable time series forecasting. The lookback window is set to 96. The results of FilterNet [21], TimeXer [33], iTransformer [18], TimeMixer [30], PatchTST [13], TSMixer [36] and MRDNet are from our experiments, and the others are reported following iTransformer[18].

Models		**MRDNet ours**		FilterNet 2024 [21]		TimeXer 2024 [33]		iTransformer 2024 [18]		TimeMixer 2024 [30]		PatchTST 2023 [36]		TSMixer 2023 [13]		Crossformer 2023 [17]		TiDE 2023 [19]		TimesNet 2023 [23]		DLinear 2022 [8]		FEDformer 2022 [12]	
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ECL	96	**0.134**	**0.230**	0.147	0.245	*0.140*	0.242	0.148	*0.240*	0.156	0.247	0.166	0.252	0.157	0.260	0.219	0.314	0.237	0.329	0.168	0.272	0.197	0.282	0.193	0.308
	192	**0.153**	**0.247**	0.160	*0.250*	*0.157*	0.256	0.164	0.256	0.171	0.261	0.174	0.260	0.173	0.274	0.231	0.322	0.236	0.330	0.184	0.289	0.196	0.285	0.201	0.315
	336	**0.171**	**0.267**	*0.173*	**0.267**	0.177	0.275	0.178	*0.270*	0.185	0.275	0.191	0.278	0.192	0.295	0.246	0.337	0.249	0.344	0.198	0.300	0.209	0.301	0.214	0.329
	720	**0.201**	**0.292**	*0.210*	0.309	*0.210*	*0.307*	0.228	0.312	0.225	0.310	0.230	0.311	0.223	0.318	0.280	0.363	0.284	0.373	0.220	0.320	0.245	0.333	0.246	0.355
Traffic	96	**0.374**	**0.257**	0.430	0.294	0.428	0.271	*0.393*	*0.269*	0.486	0.299	0.446	0.286	0.493	0.336	0.522	0.290	0.805	0.493	0.593	0.321	0.650	0.396	0.587	0.366
	192	**0.401**	**0.267**	0.452	0.307	0.448	0.282	*0.413*	*0.278*	0.495	0.301	0.453	0.285	0.497	0.351	0.530	0.293	0.756	0.474	0.617	0.336	0.598	0.370	0.604	0.373
	336	**0.421**	**0.276**	0.470	0.316	0.472	0.289	*0.425*	*0.283*	0.492	0.309	0.467	0.291	0.528	0.361	0.558	0.305	0.762	0.477	0.629	0.336	0.605	0.373	0.621	0.383
	720	**0.458**	**0.298**	0.498	0.323	0.515	0.307	*0.461*	*0.302*	0.532	0.323	0.500	0.309	0.569	0.380	0.589	0.328	0.719	0.449	0.640	0.350	0.645	0.394	0.626	0.382
Weather	96	**0.156**	**0.201**	0.162	0.207	*0.157*	*0.205*	0.175	0.216	0.164	0.210	0.177	0.218	0.166	0.210	0.158	0.230	0.202	0.261	0.172	0.220	0.196	0.255	0.217	0.296
	192	*0.205*	**0.246**	0.210	0.250	**0.204**	*0.247*	0.225	0.258	0.207	0.250	0.223	0.257	0.215	0.256	0.206	0.277	0.242	0.298	0.219	0.261	0.237	0.296	0.276	0.336
	336	**0.261**	**0.288**	0.265	*0.290*	**0.261**	*0.290*	0.280	0.298	0.262	*0.290*	0.277	0.297	0.287	0.300	0.272	0.335	0.287	0.335	0.280	0.306	0.283	0.335	0.339	0.380
	720	0.343	*0.344*	*0.342*	**0.340**	**0.340**	**0.340**	0.361	0.351	0.347	0.347	0.365	0.367	0.355	0.348	0.398	0.418	0.351	0.386	0.365	0.359	0.345	0.381	0.403	0.428
Solar-Energy	96	**0.178**	0.245	0.204	*0.242*	0.214	0.256	*0.206*	**0.240**	0.214	0.244	0.205	0.250	0.221	0.275	0.310	0.331	0.312	0.399	0.250	0.292	0.290	0.378	0.242	0.342
	192	**0.186**	**0.249**	0.239	0.269	0.238	0.275	*0.239*	*0.265*	0.252	0.286	0.232	*0.265*	0.268	0.306	0.734	0.725	0.339	0.416	0.296	0.318	0.320	0.398	0.285	0.380
	336	**0.203**	**0.259**	0.255	0.279	0.238	0.276	*0.252*	*0.275*	0.253	0.285	0.250	0.277	0.272	0.294	0.750	0.735	0.368	0.430	0.319	0.330	0.353	0.415	0.282	0.376
	720	**0.207**	**0.257**	0.258	0.282	0.244	0.284	*0.251*	*0.276*	0.240	0.269	0.255	0.285	0.281	0.313	0.769	0.765	0.370	0.425	0.338	0.337	0.356	0.413	0.357	0.427
PEMS03	12	**0.061**	**0.164**	0.072	0.179	*0.068*	*0.175*	0.069	*0.175*	0.078	0.187	0.072	0.179	0.075	0.186	0.090	0.203	0.178	0.305	0.085	0.192	0.122	0.243	0.126	0.251
	24	**0.077**	**0.185**	0.098	0.209	*0.089*	*0.199*	0.097	0.208	0.115	0.228	0.103	0.212	0.095	0.210	0.121	0.240	0.257	0.371	0.118	0.223	0.201	0.317	0.149	0.275
	48	**0.108**	**0.222**	0.145	0.258	*0.122*	*0.232*	0.137	0.247	0.182	0.288	0.160	0.266	0.121	0.240	0.202	0.317	0.379	0.463	0.155	0.260	0.333	0.425	0.227	0.348
	96	**0.139**	**0.254**	0.195	0.302	*0.162*	*0.273*	0.181	0.288	0.227	0.329	0.208	0.306	0.184	0.295	0.262	0.367	0.490	0.539	0.228	0.317	0.457	0.515	0.348	0.434
PEMS04	12	**0.066**	**0.167**	0.078	*0.186*	*0.075*	0.187	0.081	0.188	0.093	0.203	0.087	0.196	0.079	0.188	0.098	0.218	0.219	0.340	0.087	0.195	0.148	0.272	0.138	0.262
	24	**0.076**	**0.182**	0.099	0.213	*0.086*	*0.200*	0.100	0.212	0.130	0.244	0.120	0.232	0.089	0.201	0.131	0.256	0.292	0.398	0.103	0.215	0.224	0.340	0.177	0.293
	48	**0.091**	**0.202**	0.134	0.251	*0.114*	*0.229*	0.134	0.248	0.197	0.304	0.174	0.282	0.111	0.222	0.205	0.326	0.409	0.478	0.136	0.250	0.355	0.437	0.270	0.368
	96	**0.110**	**0.222**	0.166	0.281	*0.154*	*0.267*	0.172	0.284	0.250	0.346	0.220	0.323	0.133	0.247	0.402	0.457	0.492	0.532	0.190	0.303	0.452	0.504	0.341	0.427
PEMS07	12	**0.053**	**0.146**	0.063	*0.163*	*0.061*	0.166	0.067	0.165	0.071	0.175	0.070	0.173	0.083	0.181	0.094	0.200	0.173	0.304	0.082	0.181	0.115	0.242	0.109	0.225
	24	**0.067**	**0.166**	0.086	0.193	*0.074*	*0.177*	0.087	0.190	0.110	0.218	0.106	0.211	0.090	0.199	0.139	0.247	0.271	0.383	0.101	0.204	0.210	0.329	0.125	0.244
	48	**0.081**	**0.185**	0.122	0.234	0.111	0.221	*0.110*	*0.215*	0.189	0.287	0.176	0.273	0.124	0.231	0.311	0.369	0.446	0.495	0.134	0.238	0.398	0.458	0.165	0.288
	96	**0.103**	**0.206**	0.160	0.270	0.156	0.254	*0.139*	*0.245*	0.265	0.341	0.249	0.331	0.163	0.255	0.396	0.442	0.628	0.577	0.181	0.279	0.594	0.553	0.262	0.376
PEMS08	12	**0.069**	**0.168**	*0.079*	*0.182*	0.082	0.192	0.089	0.194	0.090	0.196	0.088	0.191	0.083	0.189	0.165	0.214	0.227	0.343	0.112	0.212	0.154	0.276	0.173	0.273
	24	**0.092**	**0.192**	0.114	*0.220*	*0.113*	0.222	0.137	0.242	0.133	0.237	0.135	0.236	0.117	0.226	0.215	0.260	0.318	0.409	0.141	0.238	0.248	0.353	0.210	0.301
	48	**0.139**	**0.239**	*0.180*	*0.279*	0.186	0.288	0.249	0.289	0.214	0.303	0.241	0.318	0.196	0.299	0.315	0.355	0.497	0.510	0.198	0.283	0.440	0.470	0.320	0.394
	96	**0.175**	**0.227**	0.236	0.291	0.272	0.299	*0.221*	*0.267*	0.291	0.350	0.255	0.320	0.266	0.331	0.377	0.397	0.721	0.592	0.320	0.351	0.674	0.565	0.442	0.465
ETTm1	96	**0.310**	**0.351**	0.321	0.361	*0.318*	*0.356*	0.341	0.376	0.321	0.360	0.329	0.365	0.323	0.363	0.404	0.426	0.364	0.387	0.338	0.375	0.345	0.372	0.379	0.419
	192	**0.357**	**0.376**	0.367	0.387	*0.362*	*0.383*	0.381	0.395	0.368	0.387	0.380	0.394	0.376	0.392	0.450	0.451	0.398	0.404	0.374	0.387	0.380	0.389	0.426	0.441
	336	**0.385**	**0.397**	0.401	0.409	0.395	0.407	0.419	0.419	*0.391*	*0.405*	0.400	0.410	0.407	0.413	0.532	0.515	0.428	0.425	0.410	0.411	0.413	0.413	0.445	0.459
	720	*0.453*	**0.436**	0.477	0.448	**0.452**	*0.441*	0.486	0.456	*0.453*	0.442	0.475	0.453	0.485	0.459	0.666	0.589	0.487	0.461	0.478	0.450	*0.474*	0.453	0.543	0.490
ETTm2	96	*0.172*	**0.255**	0.175	0.258	**0.171**	*0.256*	0.184	0.267	0.180	0.264	0.178	0.260	0.182	0.266	0.287	0.366	0.207	0.305	0.187	0.267	0.193	0.292	0.203	0.287
	192	**0.234**	**0.297**	0.240	0.301	*0.236*	*0.299*	0.253	0.312	0.242	0.303	0.247	0.305	0.249	0.309	0.414	0.492	0.290	0.364	0.249	0.309	0.284	0.362	0.269	0.328
	336	**0.292**	**0.336**	0.311	0.347	*0.295*	0.338	0.315	0.352	0.297	*0.337*	0.302	0.341	0.309	0.347	0.597	0.542	0.377	0.422	0.321	0.351	0.369	0.427	0.325	0.366
	720	*0.397*	*0.398*	0.414	0.405	**0.393**	**0.394**	0.412	0.406	0.399	0.404	0.407	0.401	0.416	0.408	1.730	1.042	0.558	0.524	0.408	0.403	0.554	0.522	0.421	0.415
ETTh1	96	**0.373**	**0.392**	0.382	0.402	0.382	0.403	0.386	0.405	0.378	*0.398*	0.395	0.407	0.401	0.412	0.423	0.448	0.479	0.464	0.384	0.402	0.386	0.400	*0.376*	0.419
	192	*0.427*	**0.423**	0.430	*0.429*	0.429	0.435	0.441	0.436	0.441	0.430	0.446	0.434	0.452	0.442	0.471	0.474	0.525	0.492	0.436	*0.429*	0.437	0.432	**0.420**	0.448
	336	0.477	**0.448**	0.472	0.451	*0.467*	*0.449*	0.491	0.461	0.500	0.460	0.489	0.457	0.492	0.463	0.570	0.546	0.565	0.512	0.491	0.469	0.481	0.459	**0.459**	0.465
	720	0.488	0.476	*0.481*	*0.473*	0.499	0.493	0.509	0.494	**0.479**	**0.472**	*0.488*	0.477	0.507	0.490	0.653	0.621	0.594	0.558	0.521	0.500	0.519	0.516	0.506	0.507
ETTh2	96	**0.284**	**0.338**	0.293	0.343	*0.286*	**0.338**	0.300	0.350	0.289	*0.341*	0.288	0.340	0.319	0.361	0.745	0.584	0.400	0.440	0.340	0.374	0.333	0.387	0.358	0.397
	192	*0.368*	*0.392*	0.374	0.396	**0.363**	**0.389**	0.382	0.400	0.378	0.396	0.376	0.395	0.402	0.410	0.877	0.656	0.528	0.509	0.402	0.414	0.477	0.476	0.429	0.439
	336	*0.416*	*0.429*	0.417	0.430	**0.412**	**0.424**	0.424	0.432	0.428	0.434	0.440	0.451	0.444	0.446	1.043	0.731	0.643	0.571	0.452	0.452	0.594	0.541	0.496	0.487
	720	*0.428*	*0.446*	0.449	0.460	0.448	0.453	**0.426**	**0.445**	0.468	0.468	0.436	0.453	0.441	0.450	1.104	0.763	0.874	0.679	0.462	0.468	0.831	0.657	0.463	0.474
1st Count		**37**	**41**	0	2	7	5	1	2	1	1	0	0	0	0	0	0	0	0	0	0	2	0	0	0

Table 3
Additional comparison using Dynamic Time Warping (DTW) distance [37]. The computation follows the implementation from TSLib [38].

Model	ECL				Weather				Traffic				PEMS08			
	96	192	336	720	96	192	336	720	96	192	336	720	12	24	48	96
MRDNet	**589.91**	**1261.14**	**2341.78**	**5567.63**	**141.02**	**335.30**	**674.58**	**1683.14**	**1416.18**	**2908.30**	**5177.56**	**11473.24**	**38.62**	**88.82**	**214.87**	**463.18**
TexFilter	622.63	1308.38	2421.19	5906.51	147.34	347.29	695.70	1707.60	1532.20	3098.11	5510.84	12146.01	41.31	98.41	245.08	536.51
TimeXer	609.01	1285.45	2389.78	5650.66	142.68	336.07	677.74	1678.28	1506.86	3063.63	5487.26	12164.80	42.07	96.37	233.14	590.71
iTransformer	621.93	1322.11	2413.93	5686.57	151.89	354.72	704.85	1725.89	1450.44	2956.94	5244.35	11644.95	43.21	106.14	315.65	685.75

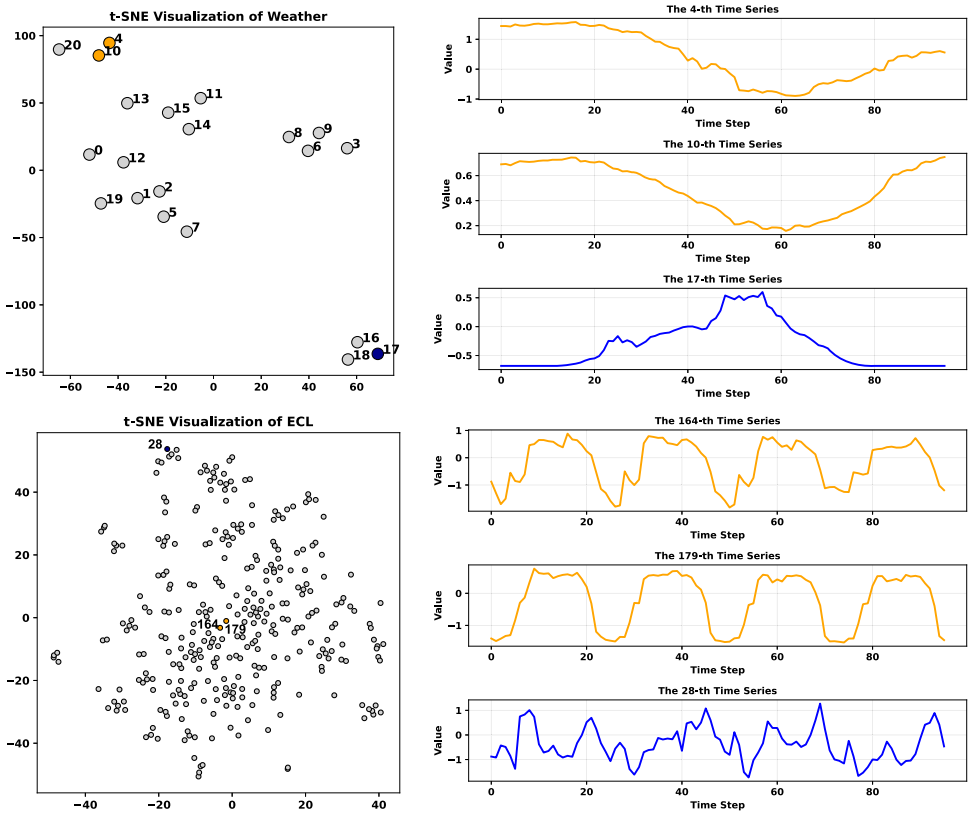

Fig. 4. T-SNE visualization of global latent vectors. In the Weather and ECL datasets, variables closer in the latent space exhibit greater similarity, while those farther apart show weaker similarity.

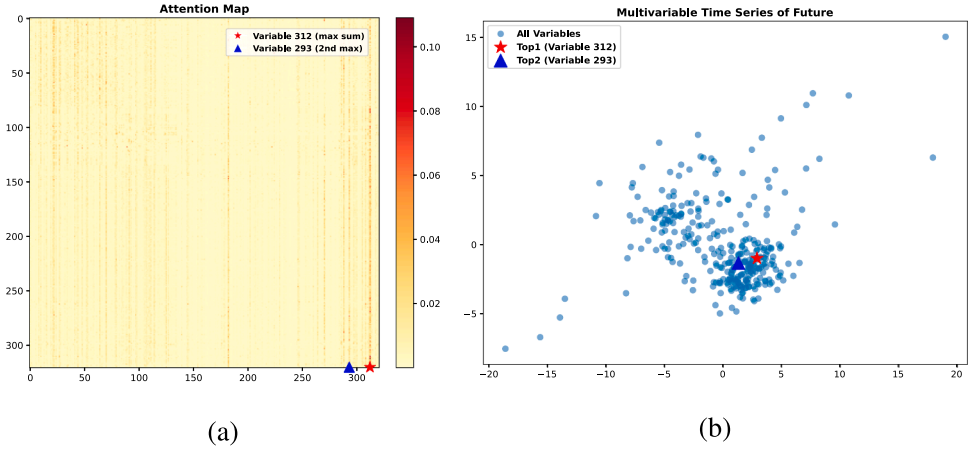

Fig. 5. (a) Heatmap of the similarity matrix after affine transformation and Softmax normalization, where variables with the highest column sums are selected as approximate central variables. (b) PCA distribution of the actual future sequence, with the central variable from the attention map coinciding with the center of the PCA distribution.

Table 4
Ablation experiments. The 'w/o GCM' represents that only Local Cross-variable Modeling (LCM) is employed to model local relationships in Seasonal and Trend branches. Additionnally, 'w/o LCM' indicates that only Global Cross-variable Modeling (GCM) is employed to model global relationships in two branches.

	ECL(MSE)				Traffic(MSE)				PEMS08(MSE)			ETTm1(MSE)				
Prediction length	96	196	336	720	96	196	336	720	12	24	48	96	96	196	336	720
iTransformer	0.148	0.164	0.178	0.228	0.393	0.413	0.425	0.461	0.089	0.137	0.249	0.221	0.341	0.381	0.419	0.486
w/o GCM	0.145	0.160	0.177	0.210	0.381	0.409	0.434	0.467	0.072	0.100	0.156	0.192	0.317	0.360	0.398	0.461
w/o LCM	0.158	0.176	0.191	0.217	0.460	0.484	0.510	0.555	0.100	0.135	0.173	0.213	0.324	0.363	0.392	0.454
MRDNet (ours)	**0.134**	**0.153**	**0.171**	**0.201**	**0.374**	**0.401**	**0.421**	**0.458**	**0.069**	**0.092**	**0.139**	**0.175**	**0.310**	**0.357**	**0.385**	**0.453**

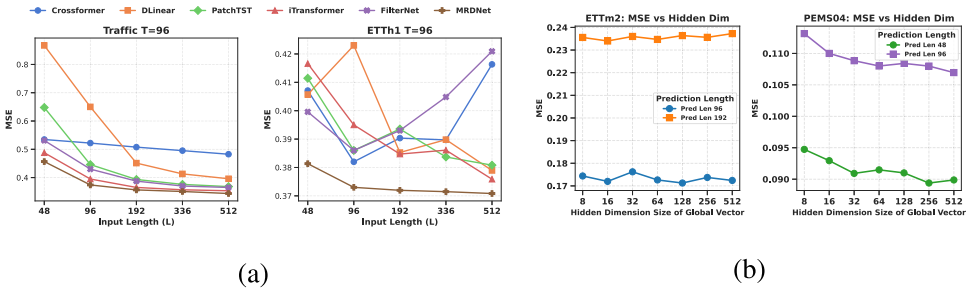

Fig. 6. (a) Evaluation of the varying input length L on the ECL, Traffic, and ETTh1 datasets, where $L \in \{48, 96, 192, 336, 512\}$ and the forecast horizon is set to $T = 96$. As L increases, MRDNet demonstrates enhanced accuracy and outperforms other models. (b) Comparison of Mean Squared Error (MSE) across different global hidden vector dimensions on the ETTm2 and PEMS08 datasets.

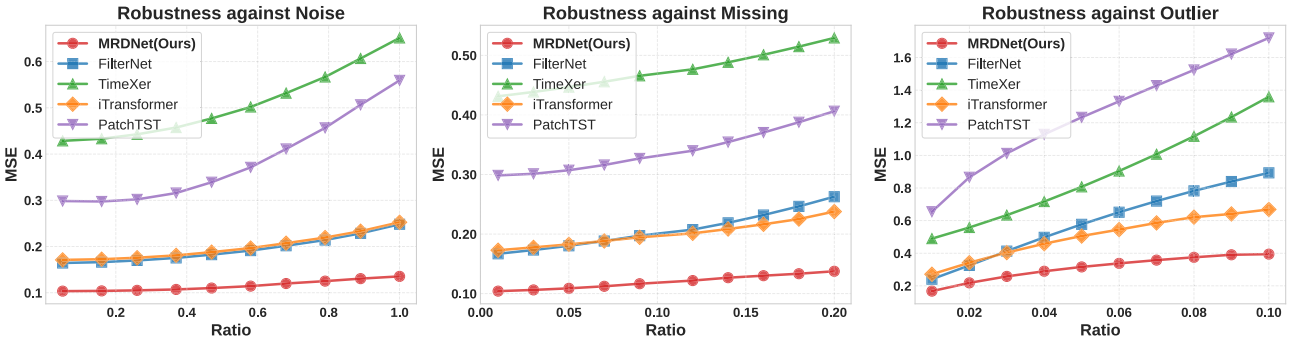

Fig. 7. Robustness analysis of MRDNet and competitive baselines under varying intensities of Gaussian Noise (left), Data Missing (middle), and Outliers (right). Results are averaged over five random seeds.

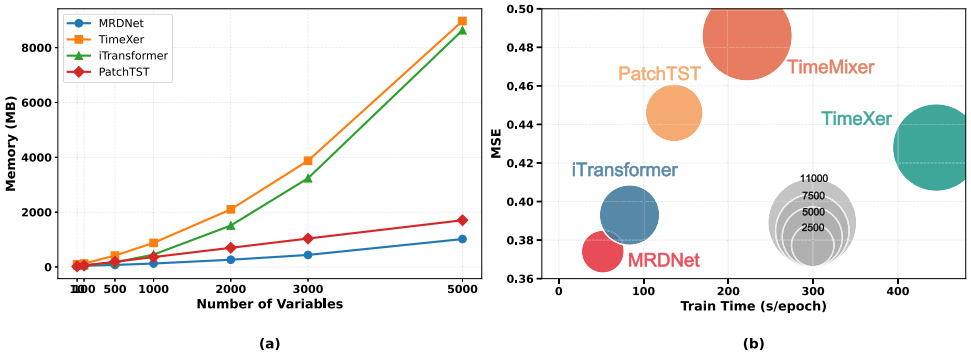

Fig. 8. Efficiency comparison of MRDNet with baseline models, including (a) memory usage versus the number of variables, and (b) training time per epoch, MSE, and peak memory usage on the Traffic dataset.

performance. Notably, the 'w/o GCM' configuration outperforms iTransformer, indicating that LCM captures local variable relationships more effectively than self-attention. This is attributed to the attention map's susceptibility to outliers, missing values, or noise, which can lead to erroneous learning, whereas LCM employs affine transformations to modify variable autocorrelations, significantly mitigating such effects. On the other hand, the 'w/o GCM' configuration yields better overall predictions than 'w/o LCM,' suggesting that capturing static global relationships alone is insufficient for addressing dynamically evolving variable interactions. Overall, MRDNet effectively captures both global and local variable relationships, resulting in enhanced predictive performance.

4.5. Parameter sensitivity analysis

4.5.1. Varing input length

Prior studies [8] have shown that the predictive performance of Transformer-based models, such as Informer [10], Autoformer [11], and FEDformer [12], declines with increasing input sequence length. This performance degradation stems from encoding multivariate time series into temporal tokens, which introduces noise from misaligned temporal representations. As the number of tokens (i.e., input length) grows, accumulated noise adversely affects forecasting accuracy. However, a longer lookback window should theoretically provide richer temporal information and more robust intervariable relationships, potentially improving predictive performance.

To evaluate MRDNet, we conducted experiments comparing it against state-of-the-art (SOTA) models, including FilterNet [21], iTransformer [18], PatchTST [13], DLinear [8], and Crossformer [17]. These experiments utilized the Traffic and ETTh1 datasets with input sequence lengths of $\{48, 96, 192, 336, 512\}$ and a fixed prediction horizon of 96 time steps. The results are shown in Fig. 6(a). Notably, Crossformer and iTransformer exhibit fluctuating or declining performance on the ETTh1 dataset as input length increases, likely due to redundancy in the self-attention mechanism when capturing intervariable dependencies. In contrast, MRDNet consistently improves forecasting accuracy with longer input sequences. This can be attributed to its strategy, which focuses on learning global representations and central relationships, yielding stronger robustness in noisier sequences. Furthermore, MRDNet achieves

a lower MSE across all tested input lengths compared to other SOTA models, demonstrating its ability to effectively capture stable intervariable relationships and enhance predictive performance for extended input sequences.

4.5.2. Hidden dimensions of global vector

We conducted experiments to evaluate the impact of the hidden dimension size of the Global Vector in the GCM module on prediction performance. We selected the ETTm2 and PEMS04 datasets as benchmarks, with an input length of 96 and prediction lengths of {96, 192} for ETTm2 and {48, 96} for PEMS04. The hidden dimension sizes tested were {8, 16, 32, 64, 128, 256, 512}. Results, shown in Fig. 6(b), indicate that on the ETTm2 dataset, prediction performance fluctuates with increasing hidden dimensions, likely due to overfitting caused by the dataset's limited variable count. Consequently, a hidden dimension of 16 is recommended for datasets with fewer variables. Conversely, on the PEMS04 dataset, higher hidden dimensions correlate with lower MSE, suggesting that datasets with more variables require larger hidden dimensions (e.g., 128 or 256) to capture intrinsic variable representations effectively.

4.6. Robustness analysis

To assess model robustness in unpredictable real-world environments, we conduct a comprehensive experiment by introducing three types of perturbations: Gaussian Noise, Data Missing, and Outliers with varying intensities. To ensure statistical significance, all results are reported as the average of five runs using different random seeds.

Fig. 7 compares the performance of MRDNet against state-of-the-art baselines. We observe that patch-based models, such as PatchTST [13] and TimeXer [33], exhibit severe performance degradation as perturbation intensity increases. Since these models rely on local patches, any anomaly within a patch distorts local semantic information, leading to rapid error accumulation and severe collapse under high noise levels. While iTransformer [18] improves robustness by capturing correlations across the entire input window, its ability to maintain precise variable dependencies remains compromised in high-perturbation scenarios.

In contrast, MRDNet demonstrates remarkable stability across all categories. Notably, our model's error rate at maximum perturbation intensity remains lower than those of baselines under significantly milder conditions. This superior robustness stems from our dual-interaction architecture. By explicitly decomposing variable relations into global and local components, MRDNet effectively filters out transient local distortions while maintaining focus on underlying global patterns or central dependencies. This design ensures that the model captures authentic cross-variable dependencies even in highly corrupted environments, achieving consistent and reliable forecasting.

4.7. Efficiency analysis

We conducted two experiments to evaluate the efficiency of our proposed MRDNet. In the first experiment, we varied the number of variables while keeping the batch size fixed at 1 and both the input and prediction lengths at 96. As shown in Fig. 8 (a), MRDNet consistently consumes less memory than the baseline models as the number of variables increases. This advantage comes from the fact that, although the LCM module involves an $N \times N$ Gram matrix, we use only a single layer without multiple attention heads. As a result, the computational overhead remains low. These results highlight the strong scalability of MRDNet, making it well-suited for complex real-world time series forecasting tasks with many variables.

In the second experiment, we compared the models on the Traffic dataset using a batch size of 16 (to ensure fair comparison) and input/prediction lengths of 96. All experiments were run on an NVIDIA A6000 GPU with 48 GB memory. The results are shown in Fig. 8 (b). Compared to mainstream baselines, MRDNet achieves a better overall trade-off between efficiency and performance. In particular, MRDNet requires significantly less training time because it computes only a single Gram matrix layer.

5. Conclusion

The Multivariate Relational Decomposition Network (MRDNet) effectively addresses the challenges of modeling multivariate time series data in the presence of anomalies such as outliers, missing values, and noise. By decomposing variable relationships into global and local components, MRDNet captures stable, long-term seasonal patterns and dynamic, window-specific trends, while mitigating the adverse effects of anomalies through adaptive neural refinement. The integration of these components into a unified architecture enables MRDNet to achieve robust and accurate forecasting performance. Extensive experiments on benchmark datasets validate MRDNet's superior ability to handle complex multivariate correlations, making it a powerful tool for enhancing the reliability and precision of time series predictions across diverse applications.

CRediT authorship contribution statement

Ao Hu: Writing – original draft, Software, Resources; **Liangjian Wen:** Supervision, Project administration; **Mingyi Zhang:** Software, Project administration; **Yong Dai:** Validation, Supervision, Conceptualization; **Dongkai Wang:** Formal analysis, Data curation; **Jun Wang:** Validation, Supervision; **Jiang Duan:** Writing – review & editing, Project administration.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declared that they have no conflicts of interest to this work. We declare that we do not have any commercial or associative interest that represents a conflict of interest in connection with this work.

Acknowledgement

This work was supported by the Major Science and Technology Special Project of the Sichuan Provincial Department of Science and Technology (Grant No. 2024ZDZX0002), the Sichuan Provincial Innovation Group Project (Grant No. 2024NSFTD0054), Fundamental Research Funds for the Central Universities (JBK202511081), the Blockchain Research Center of China, the Natural Science Foundation of China (Grant No. 62502397), the National Natural Science Foundation of China (72471197), and the Sichuan Provincial Philosophy and Social Science Fund (Grant No. SCJJ25ND091).

References

[1] L. Wen, Q. Hu, C. Guo, A. Hu, M. Zhang, Cross-scale attention for long-term time series forecasting, IEEE Signal Process. Lett., 31 (2024) 2675–2679 .
[2] M. Usman, Y. Lee, DFDG: Adaptive federated learning for dynamic graph-based traffic forecasting, Knowl. Syst. 326 (2025) 114019.
[3] S. Mehtab, S. Jaydip, A time series analysis-based stock price prediction using machine learning and deep learning models, Int. J. Bus. Forecasting Mark. Intell. 6 (2020) 272.
[4] J. Liu, J. Guo, L. Gao, Y. Wang, A. Liu, X. Zhang, PPTformer: a novel hybrid model for enhanced long-term time series forecasting with extreme value focus, Knowl. Syst. 317 (2025) 113456.
[5] E.N. Lorenz, S.F. P. M. I.o. Technology), Empirical Orthogonal Functions and Statistical Weather Prediction, 1956.
[6] H. Zhang, J. Wang, Y. Nie, A novel optimization model based on fuzzy time series for short-term air quality index forecasting, Knowl. Syst. 296 (2024) 111905.
[7] J. Deng, X. Chen, R. Jiang, X. Song, I.W. Tsang, ST-Norm: spatial and temporal normalization for multi-variate time series forecasting, in: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, 2021.

[8] A. Zeng, M. Chen, L. Zhang, Q. Xu, Are transformers effective for time series forecasting?, AAAI 37 (2023) 11121–11128.
[9] M. Liu, A. Zeng, M. Chen, Z. Xu, Q. Lai, L. Ma, Q. Xu, SCINet: time series modeling and forecasting with sample convolution and interaction, NeurIPS 35 (2022) 5816–5828.
[10] J. Li, X. Hui, W. Zhang, Informer: beyond efficient transformer for long sequence time-series forecasting, AAAI 35 (2021) 11106–11115.
[11] H. Wu, J. Xu, J. Wang, M. Long, Autoformer: decomposition transformers with auto-correlation for long-term series forecasting, NeurIPS 34 (2021) 22419–22430.
[12] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, R. Jin, FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting, ICML 162 (2022) 27268–27286.
[13] Y. Nie, N.H. Nguyen, P. Sinthong, J. Kalagnanam, A time series is worth 64 words: long-term forecasting with transformers, ICLR (2023).
[14] B. Yu, H. Yin, Z. Zhu, Spatio-temporal graph convolutional networks: a deep learning framework for traffic forecasting, IJCAI (2018) 3634–3640.
[15] Z. Wu, S. Pan, G. Long, J. Jiang, X. Chang, C. Zhang, Connecting the dots: multivariate time series forecasting with graph neural networks, Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (2020) 753–763.
[16] Q. Huang, L. Shen, R. Zhang, S. Ding, B. Wang, Z. Zhou, Y. Wang, CrossGNN: confronting noisy multivariate time series via cross interaction refinement, NeurIPS 36 (2023) 46885–46902.
[17] Y. Zhang, J. Yan, Crossformer: transformer utilizing cross-dimension dependency for multivariate time series forecasting, ICLR (2023).
[18] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, M. Long, iTransformer: inverted transformers are effective for time series forecasting, ICLR (2024).
[19] A. Das, W. Kong, A. Leach, S. Mathur, S. Rajat, R. Yu, Long-term forecasting with TiDE: time-series dense encoder, 2024, arXiv:2304.08424.
[20] Z. Li, S. Qi, Y. Li, Z. Xu, Revisiting long-term time series forecasting: an investigation on linear mapping, 2023, arXiv:2305.10721.
[21] K. Yi, J. Fei, Q. Zhang, H. He, S. Hao, D. Lian, W. Fan, FilterNet: harnessing frequency filters for time series forecasting, 37 55115–55140.
[22] H. Wang, J. Peng, F. Huang, J. Wang, J. Chen, Y. Xiao, MICN: multi-scale local and global context modeling for long-term series forecasting (2023).
[23] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, M. Long, TimesNet: temporal 2D-variation modeling for general time series analysis, ICLR (2023).
[24] D. Luo, X. Wang, ModernTCN: a modern pure convolution structure for general time series analysis, ICLR (2024).
[25] S.J. Taylor, B. Letham, Forecasting at scale, Am. Stat. 72 37–45.
[26] B.N. Oreshkin, D. Carpov, N. Chapados, Y. Bengio, N-BEATS: neural basis expansion analysis for interpretable time series forecasting, 2019, arXiv:1905.10437.
[27] S. Rajat, H.-F. Yu, I.S. Dhillon, Think globally, act locally: a deep neural network approach to high-dimensional time series forecasting, Adv. Neural Inf. Process. Syst., 32 4838–4847.
[28] Z. Pan, Y. Jiang, S. Garg, A. Schneider, Y. Nevmyvaka, D. Song, S^2IP-LLM: semantic space informed prompt learning with LLM for time series forecasting, in: Forty-first International Conference on Machine Learning, 2024.
[29] S. Liu, H. Yu, C. Liao, J. Li, W. Lin, A.X. Liu, S. Dustdar, Pyraformer: low-complexity pyramidal attention for long-range time series modeling and forecasting, Int. Conf. Learn. Representations (2021).
[30] S. Wang, H. Wu, X. Shi, T. Hu, H. Luo, L. Ma, J.Y. Zhang, J. ZHOU, TimeMixer: decomposable multiscale mixing for time series forecasting, in: International Conference on Learning Representations (ICLR), 2024.
[31] K. Yi, Q. Zhang, W. Fan, S. Wang, P. Wang, H. He, N. An, D. Lian, L. Cao, Z. Niu, Frequency-domain MLPs are more effective learners in time series forecasting, in: Thirty-seventh Conference on Neural Information Processing Systems, 2023.
[32] X. Zhijian, Z. Ailing, X. Qiang, FITS: modeling time series with 10k parameters, ICLR (2024).
[33] Y. Wang, H. Wu, J. Dong, Y. Liu, Y. Qiu, H. Zhang, J. Wang, M. Long, Timexer: empowering transformers for time series forecasting with exogenous variables, Adv. Neural Inf. Process. Syst. (2024).
[34] R.B. CLEVELAND, STL: a seasonal-trend decomposition procedure based on loess, J. Off. Stat. (1990).
[35] G. Lai, W.-C. Chang, Y. Yang, H. Liu, Modeling long-and short-term temporal patterns with deep neural networks, SIGIR (2018) 95–104.
[36] V. Ekambaram, A. Jati, N. Nguyen, P. Sinthong, J. Kalagnanam, Tsmixer: lightweight mlp-mixer model for multivariate time series forecasting, in: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2023.
[37] H. Sakoe, S. Chiba, Dynamic programming algorithm optimization for spoken word recognition, IEEE Trans. Acoust.,26 43–49.
[38] Y. Wang, H. Wu, J. Dong, Y. Liu, M. Long, J. Wang, Deep time series models: a comprehensive survey and benchmark, 2024, arXiv:2407.13278.